

Data & Scheduling Architecture

Medellín Social — R2 data tiering and GHL as the realtor-facing calendar

Prepared 2026-08-18 · Decision-support document, not an executed migration plan

Contents

- 1. Current state
- 2. Why "move the whole database to R2" doesn't work
- 3. What actually can move: a per-schema strategy
- 4. GHL as the realtor's calendar and payment surface
- 5. Implementation plan
- 6. Risks and mitigations
- 7. Open decisions

1. Current state

Component	Where it runs today	Nature
Frontend (TanStack Start)	Railway, Dockerfile	Stateless HTTP
API (FastAPI)	Railway, Dockerfile.api	Stateless HTTP
urbi - cron	Railway, Dockerfile.cron — always-on container with Chromium baked in	Scheduled scraping: fincaraiz + remates daily, community scrapers every 6h
Airflow pipeline (Metrocuadrado, dedup, R2 media mirror, traffic)	Local, developer's WSL2 machine	Heavier scheduled batch — not in the cloud today
PostgreSQL + PostGIS	Railway — self-hosted postgis/postgis Docker image (not Railway's managed Postgres)	Stateful, 1.5 GB total
Listing photos	Cloudflare R2	Object storage — already correct, backfill pending
Payments / checkout links	GoHighLevel (GHL)	SaaS, external — we redirect to their checkout

Database size by schema

Schema	Size	Content
raw	1.5 GB	Raw scraped snapshots (Fincaraíz, Metrocuadrado, cadastral records) — append-mostly, rarely queried after processing
staging	153 MB	ETL intermediate tables, rebuilt on every cache refresh
analytics	43 MB	Computed/derived neighborhood scores — read-heavy, batch-written
public	54 MB	Operational data: listings, agents, agencies, users, visit requests — transactional, foreign keys, PostGIS geometry on every request

2. Why "move the whole database to R2" doesn't work

R2 is Cloudflare's S3-compatible object storage — it stores files/blobs. It has no query engine of its own, no foreign key enforcement, no row-level transactions, and no PostGIS.

PostGIS spatial functions are load-bearing across the application: 36 backend files call ST_Contains/ST_DWithin, including the agent-zone assignment logic that decides who gets paid for a lead in a given comuna/barrio (asignador_service.py, zona_service.py), and every listings/neighborhood query (cache.py, listings.py, comunidad.py, business.py). None of that logic exists without a real spatial SQL engine sitting behind live requests.

Cost math doesn't support a full migration either. The entire database is 1.5 GB. A Postgres instance that size costs roughly \$1–3/month on Railway. Moving it to R2 would save single-digit dollars, not meaningfully change the bill.

3. What actually can move: a per-schema strategy

The database is not one workload — it's three, with different requirements. Two new tools make a partial move genuinely viable:

- **DuckDB's spatial extension** implements ~100 ST_* functions modeled on PostGIS, and reads Parquet files directly over HTTP from any S3-compatible endpoint (R2 included) via its httpfs extension.
- **Cloudflare R2 SQL + R2 Data Catalog** is Cloudflare's own managed serverless SQL engine over Apache Iceberg tables stored in R2 — ACID transactions at the table level, zero egress fees, purpose-built for analytics/BI workloads (not row-by-row OLTP).

Schema	Recommendation	Rationale
--------	----------------	-----------

raw (1.5 GB)	Move to R2 as Parquet, queried via DuckDB	Append-only, rarely touched after the nightly refresh consumes it. Biggest schema by far — this is where the storage savings actually live.
analytics (43 MB)	Move to R2 via R2 Data Catalog + R2 SQL	Read-heavy, batch-written by scoring jobs, no live transactional writes — exactly the workload Cloudflare designed this for.
staging (153 MB)	Stays in Postgres	Rebuilt on every cache refresh from spatial joins against public — needs to sit next to the live geometry data it joins against.
public (54 MB)	Stays in Postgres	Transactional core: foreign keys, live writes, PostGIS on every user-facing request. No object-storage query engine gives per-row ACID transactions at web-request latency.

raw + analytics together are 1.54 GB of the database's 1.6 GB total — the large majority by weight can move. What stays in Postgres is small specifically because it's transactional, not because it's unimportant.

What this costs to build

1. Rewrite `cache.py`'s `_REFRESH_STG_LISTINGS` step to read `raw.*` via DuckDB against Parquet in R2 instead of a plain SQL `INSERT ... SELECT ... FROM raw.*`.
2. Scrapers that currently write to `raw.*` tables switch to writing Parquet files to R2.
3. Batch jobs that populate `analytics.*` (the `compute_*.py` scripts) write Parquet/Iceberg to R2 instead of INSERTing into Postgres; API endpoints that read `analytics.*` switch to R2 SQL queries.

This is a real engine swap for two of three schemas — not a configuration change — but it uses off-the-shelf tools for exactly this shape of problem, not custom infrastructure.

4. GHL as the realtor's calendar and payment surface

Separately, the business wants realtors to manage their schedule and receive payment inside GHL — where they already work daily — rather than exclusively in the app's own dashboard.

Constraint that shapes the design

Medellín Social operates a single GHL sub-account with each agent as a team member with their own calendar inside it (confirmed — not one GHL account per realtor). This keeps the integration to one API key for the whole business, not per-tenant credentials.

Recommended pattern: dual-write, GHL is realtor-facing, Postgres stays the analytics source of truth

Going 100% GHL with no local copy would break an already-built, working feature: `/realtor/desempeno` (acceptance rate, show rate, offers in the last 90 days, ROI per sponsored zone) is computed today from `visita_solicitud` rows in Postgres. GHL's reporting will not have the app's custom fields (e.g. the "interested / offer / discarded" outcome tracking already built for the realtor CRM). Losing that dashboard is the concrete cost of a GHL-only design — avoidable at the same build cost.

Touchpoint	Behavior
GET <code>/listings/{id}/slots</code> (read availability)	Call GHL's calendar free-slots API live for the zone-owning agent's <code>ghl_calendar_id</code> , with a short cache (~60s) and a fallback to the existing local <code>agent_disponibilidad</code> computation if GHL is slow or unavailable.
POST <code>/visitas</code> (book a visit)	Insert into <code>visita_solicitud</code> as today (unchanged — this keeps the dashboard alive), <i>and</i> create the appointment in GHL, best-effort (same non-blocking try/except pattern already used for lead-assignment emails). Store the returned GHL appointment id on the row for later updates/cancellations.
PATCH <code>/realtor/visitas/{id}</code> (status change)	Propagate the status change to the linked GHL appointment via its stored id.

Schema changes required

```
ALTER TABLE agent
  ADD COLUMN ghl_calendar_id text;

ALTER TABLE visita_solicitud
  ADD COLUMN ghl_appointment_id text;
```

Why not call GHL live on every slot request with no fallback and no cache: the public listing page is unauthenticated, high-traffic, and today fully independent of any third party. A GHL outage or a traffic spike against GHL's rate limit (100 requests/10s, 200,000/day per resource) would otherwise take down the booking flow for every visitor, not just realtors.

5. Implementation plan

1. **Instrumentation first, regardless of the rest.** Neither the R2 tiering decision nor the GHL cutover can be evaluated without real traffic and cost numbers. There is currently no APM, no aggregated request logging, and no habit of checking Railway's billing dashboard. This is the actual prerequisite for making the next two workstreams data-driven instead of speculative.
2. **GHL calendar sync** — smaller, self-contained, unblocks a real product need (payments + calendar consolidation for realtors) as soon as a GHL API key exists.
 1. Obtain the GHL API key / location credentials.

2. Add `ghl_calendar_id` to `agent`, backfill for existing active agents.
 3. Build `api/utils/ghl_client.py` (outbound: create/update appointment, read free slots) following the existing `wompi_client.py` pattern — feature-inert until the API key is set.
 4. Wire the three touchpoints in §4, each with the fallback/cache/best-effort guards specified.
3. **R2 data tiering** — larger, higher engineering cost, not time-sensitive. Sequence by risk, smallest first:
1. Start with `analytics` (43 MB, no live writes) as a pilot: export the existing batch jobs' output to R2 via R2 Data Catalog, point one read-only endpoint at R2 SQL, compare correctness and latency against the current Postgres-backed endpoint before cutting over.
 2. Only after that pilot is verified in production, move `raw` (1.5 GB, the real storage-cost driver) — rewrite the nightly `cache.py` refresh step to read from R2 via DuckDB.

6. Risks and mitigations

Risk	Mitigation already designed in
GHL outage breaks public booking	Fallback to local <code>agent_disponibilidad</code> slot computation
GHL rate limit hit under normal map traffic	60s cache on slot-availability calls
Losing realtor performance analytics	Dual-write — Postgres keeps receiving every visit row regardless of GHL's state
R2 SQL / DuckDB correctness gap vs. current PostGIS queries	Pilot on <code>analytics</code> first, compare outputs before touching <code>raw</code> or any transactional path
Decisions made on guesswork instead of real cost/traffic data	Instrumentation is sequenced first, not last

7. Open decisions

Does GHL become the sole source of scheduling configuration? If realtors configure their availability directly in GHL, the app's own "Horario" tab (`agent_disponibilidad`, already built) becomes redundant — two places to configure the same thing risk drifting out of sync. Retire it once every active agent has a configured GHL calendar, or keep it permanently as the fallback path (not just a transition measure)? This is a product decision, not a technical one.

How much does the current Railway/Postgres setup actually cost per month, and what is real traffic? Every recommendation in this document that depends on "is this worth the engineering cost" is currently an estimate. Pulling the real numbers from the Railway billing dashboard is a five-minute task that changes how urgently any of this should be prioritized.

